مدل‌سازی شناختی

جزوه‌ی کامل درس

هفته‌ی ۱۱: روش‌های Monte Carlo و الگوریتم Metropolis–Hastings

فهرست مطالب

۱ برآورد پارامتر با MCMC **۱**

۱.۱ هفته‌ی ۱۱: روش‌های Monte Carlo و الگوریتم Metropolis–Hastings ۱

۱.۱.۱ واژه‌های کلیدی ۱

۱.۱.۲ اهداف یادگیری ۲

۱.۱.۳ مقدمه ۳

۱.۱.۴ جزوه‌ی درس ۳

۱.۱.۵ مثال حل‌شده ۷

۱.۱.۶ lab (۲ ساعت) ۹

۱.۱.۷ ارتباط با هوش مصنوعی و LLMها ۱۲

۱.۱.۸ اشتباهات رایج و برداشت‌های نادرست ۱۲

۱.۱.۹ خودآزمایی ۱۳

۱.۱.۱۰ تکلیف ۱۳

۱.۱.۱۱ مطالعه‌ی بیشتر ۱۳

فصل ۱

برآورد پارامتر با MCMC

۱.۱ هفته‌ی ۱۱: روش‌های Monte Carlo و الگوریتم Metropolis–Hastings

فصل ۵: برآورد پارامتر با روش‌های داده‌محور (MCMC) · درس ۱ ساعت · lab ۲ ساعت **داده‌ی اصلی:** داده‌های فراموشی هفته‌ی ۲ با پیشین‌های هفته‌ی ۱۰، تا بتوانیم نمونه‌گیر را با پسین شبکه‌ای‌ای که از قبل داریم مقایسه کنیم.

۱.۱.۱ واژه‌های کلیدی

واژه‌های مهم این درس با معادل فارسی‌شان (حتی اگر در درس‌های قبل آمده باشند).

English	فارسی	توضیح کوتاه
Monte Carlo	Monte Carlo (مونت‌کارلو)	تقریب با میانگین نمونه‌های تصادفی
Metropolis algorithm	الگوریتم Metropolis	پیشنهاد یک حرکت و پذیرش یا رد آن
parameter	پارامتر	کمیت قابل تنظیم مدل که بین افراد یا شرایط فرق می‌کند
estimator / estimate	برآورد / برآوردگر	مقدار به‌دست‌آمده برای پارامتر / روش به دست آوردن آن
Markov chain Monte Carlo (MCMC)	زنجیره‌ی مارکوف Monte Carlo	نمونه‌گیری از پسین با یک زنجیره‌ی تصادفی
forgetting curve	منحنی فراموشی	کاهش یادآوری با گذشت زمان
prior	توزیع پیشین (prior)	باور درباره‌ی پارامتر پیش از دیدن داده
sampler	نمونه‌گیر (sampler)	الگوریتمی که از توزیع نمونه می‌گیرد
grid approximation	تقریب شبکه‌ای (grid)	محاسبه‌ی پسین روی شبکه‌ای از مقادیر
posterior	توزیع پسین (posterior)	باور درباره‌ی پارامتر بعد از دیدن داده

English	فارسی	توضیح کوتاه
chain	زنجیره	دنباله‌ی نمونه‌های MCMC
proposal	پیشنهاد (proposal)	مقدار جدیدی که الگوریتم امتحان می‌کند
acceptance rate	نرخ پذیرش	نسبت پیشنهادهای پذیرفته‌شده
trace plot	نمودار trace	مقدار نمونه‌ها در برابر شماره‌ی تکرار
autocorrelation	خودهمبستگی	وابستگی نمونه‌های پشت سر هم
warm-up / burn-in	دوره‌ی گرم‌شدن (burn-in)	نمونه‌های اولیه که دور ریخته می‌شوند
sensitivity (d′)	حساسیت	فاصله‌ی توزیع علامت و نویز
likelihood	درست‌نمایی	احتمال داده به‌عنوان تابعی از پارامترها
starting point	نقطه‌ی شروع	سوگیری اولیه به سمت یکی از پاسخ‌ها
response bias	سوگیری پاسخ	تمایل به یک پاسخ، مستقل از محرک
large language model (LLM)	مدل بزرگ زبانی	شبکه‌ی عصبی‌ای که کلمه‌ی بعدی متن را پیش‌بینی می‌کند
self-consistency	self-consistency (رأی اکثریت)	چند بار نمونه‌گیری جواب و انتخاب رأی اکثریت
simulation	شبیه‌سازی	تولید داده‌ی مصنوعی با اجرای مدل
MCSE	خطای استاندارد Monte Carlo	خطای شبیه‌سازی در میانگین پسین

۲.۱.۱ اهداف یادگیری

در پایان این هفته، دانشجویان می‌توانند:

۱. **توضیح دهند** چرا کمیت‌های پسین انتگرال‌اند، و چطور نمونه‌گیری Monte Carlo انتگرال‌ها را به میانگین تبدیل می‌کند، با خطایی که مثل $1/\sqrt{S}$ کم می‌شود.
۲. منطق Markov chain Monte Carlo را **توصیف کنند**: زنجیره‌ای که توزیع بلندمدتش همان پسین است و فقط از پسین نرمال‌نشده استفاده می‌کند.
۳. الگوریتم Metropolis (و Metropolis–Hastings) را برای یک مدل شناختی یک‌پارامتری و دوپارامتری از صفر **پیاده‌سازی کنند**.
۴. توزیع پیشنهادی (proposal) را با استفاده از نرخ پذیرش، trace plot و خودهمبستگی **تنظیم کنند** و دوره‌ی burn-in را **مشخص کنند**.
۵. خلاصه‌های پسین و کمیت‌های مشتق‌شده (مثلاً نیمه‌عمر) را از نمونه‌ها **حساب کنند** و نمونه‌گیر را با یک جواب معلوم **اعتبارسنجی کنند**.

۳.۱.۱ مقدمه

هر چیزی که از یک پسین می‌خواهیم یک **انتگرال** است: میانگین پسین $\int \theta\, p(\theta \mid \mathbf{y})\, d\theta$، پسین حاشیه‌ای (با انتگرال‌گیری روی بقیه‌ی پارامترها)، $P(d' > 1 \mid \mathbf{y})$، توزیع پیش‌بینی پسین، و marginal likelihood. در هفته‌های ۹–۱۰ این انتگرال‌ها را روی شبکه حساب کردیم. این برای دو پارامتر کار کرد. یک مدل SDT سلسله‌مراتبی برای ۲۰ شرکت‌کننده (هفته‌ی ۱۳) بیش از ۴۰ پارامتر دارد؛ شبکه‌ای با فقط ۱۰ نقطه برای هر پارامتر به 10^{40} ارزیابی نیاز دارد.

روش‌های Monte Carlo انتگرال‌گیری را با **نمونه‌گیری** جایگزین می‌کنند: اگر بتوانیم مقادیر زیادی از پسین بکشیم، هر انتگرالی به میانگین روی نمونه‌ها تبدیل می‌شود و دقت به تعداد نمونه‌ها بستگی دارد، نه به تعداد پارامترها. مشکل این است که نمی‌توانیم مستقیماً از پسین نمونه بگیریم؛ معمولاً حتی ثابت نرمال‌سازش را هم نمی‌توانیم حساب کنیم.

Markov chain Monte Carlo (MCMC) این مشکل را با ترفندی به‌طرز زیبایی ساده حل می‌کند: یک گام‌زنی تصادفی در فضای پارامتر که به سمت مقادیر محتمل‌تر بیشتر از مقادیر کم‌احتمال‌تر حرکت می‌کند، دقیقاً با نسبت‌های درست. بعد از مدتی، جاهایی که این گام‌زن به آن‌ها سر می‌زند نمونه‌هایی از پسین‌اند. شناخته‌شده‌ترین نسخه، یعنی **الگوریتم** Metropolis، در ۱۵ خط Python جا می‌شود و امروز پیش از استفاده از هر کتابخانه‌ای خودتان آن را می‌نویسید.

این موضوع کجا به کار می‌رود؟

- **همه‌ی نرم‌افزارهای بیزی امروزی** (PyMC، Stan، JAGS، BUGS) روی MCMC ساخته شده‌اند. هر مدل سلسله‌مراتبی در مقاله‌های اخیر روان‌شناسی و علوم اعصاب با نسخه‌ای از آن برازش شده است.
- **فیزیک و شیمی** (جایی که الگوریتم Metropolis در سال ۱۹۵۳ برای شبیه‌سازی ذرات ابداع شد)، **همه‌گیرشناسی** (برآورد پارامترهای سرایت)، **تبارشناسی** (بازسازی درخت‌های تکاملی) و **مالی** همه به MCMC تکیه دارند.
- **علوم شناختی نمونه‌گیری را به‌عنوان نظریه‌ای درباره‌ی ذهن هم به کار می‌برد:** بعضی پژوهشگرها پیشنهاد می‌کنند آدم‌ها استنباط بیزی را با نمونه‌گیری ذهنی چند امکان تقریب می‌زنند، که بعضی سوگیری‌ها را توضیح می‌دهد.
- **هوش مصنوعی:** نمونه‌گیری جواب از یک مدل زبانی همان Monte Carlo است (بخش هوش مصنوعی و LLMها را ببینید).

۴.۱.۱ جزوه‌ی درس

به زبان ساده هر سؤالی که از posterior می‌پرسیم («مقدار میانگین چیست؟»، «احتمال اینکه $d' > 1$ باشد چقدر است؟») یک انتگرال است، و grid نمی‌تواند انتگرال روی پارامترهای زیاد را حساب کند. Monte Carlo انتگرال را با میانگین جایگزین می‌کند: اگر تعداد زیادی مقدار تصادفی کشیده‌شده از posterior داشتیم، میانگین آن‌ها میانگین posterior بود، کسری که بالای ۱ است $P(d' > 1)$ بود، و الی آخر. مشکل به دست آوردن این نمونه‌هاست. **الگوریتم** Metropolis آن‌ها را با یک قدم‌زدن تصادفی هوشمندانه به دست می‌آورد. روی یک مقدار پارامتر بایستید، یک قدم تصادفی کوچک پیشنهاد کنید و مقایسه کنید جای جدید چقدر معقول است (درست‌نمایی × prior). اگر معقول‌تر است، به آنجا بروید. اگر کمتر معقول است، فقط گاهی بروید، با احتمالی برابر نسبت معقول بودن‌ها؛ وگرنه همان‌جا بمانید و مقدار فعلی را دوباره ثبت کنید. اگر این کار را به اندازه‌ی کافی ادامه دهید، جاهایی که رفته‌اید نمونه‌ای از posterior هستند. نکته‌ی جالب این است که هرگز به ثابت نرمال‌سازی سخت نیاز ندارید، چون فقط از نسبت‌ها استفاده می‌شود.

زمان‌بندی پیشنهادی (۶۰ دقیقه): انتگرال‌گیری Monte Carlo ۱۲ · زنجیره‌های مارکوف ۸ · الگوریتم Metropolis ۱۸ · Metropolis–Hastings ۵ · تنظیم، burn-in و خودهمبستگی ۱۲ · پیش‌نمایش ۵.

۱. انتگرال‌گیری Monte Carlo

اگر $\theta^{(1)}, \dots, \theta^{(S)}$ نمونه‌های مستقلی از $p(\theta \mid \mathbf{y})$ باشند، برای هر تابع g:

$$\mathbb{E}[g(\theta) \mid \mathbf{y}] = \int g(\theta)\, p(\theta \mid \mathbf{y})\, d\theta \;\approx\; \frac{1}{S}\sum_{s=1}^{S} g\big(\theta^{(s)}\big).$$

آنچه می‌خواهیم	$g(\theta)$ را انتخاب کنید	برآورد Monte Carlo
میانگین پسین	θ	میانگین نمونه‌ها
$P(\theta > 1 \mid \mathbf{y})$	شاخص $\mathbb{1}[\theta > 1]$	نسبت نمونه‌های بالای ۱
پسین یک کمیت مشتق‌شده، مثلاً نیمه‌عمر $2^{1/d}-1$	$g(d)$	g را روی هر نمونه اعمال و بعد خلاصه کنید
credible interval	—	صدک‌ها (یا HDI) نمونه‌ها
پسین حاشیه‌ای θ_1	—	فقط ستون‌های دیگر را نادیده بگیرید

خطای استاندارد Monte Carlo میانگین برای نمونه‌های مستقل $\mathrm{SD}(g)/\sqrt{S}$ است، مستقل از تعداد پارامترها. به همین دلیل نمونه‌گیری در ابعاد بالا بر شبکه برتری دارد.

خواندن ریاضی

- $\mathbb{E}[g(\theta) \mid \mathbf{y}] = \int g(\theta)\, p(\theta \mid \mathbf{y})\, d\theta$: میانگین posterior یک کمیت $g(\theta)$. انتخاب g یعنی انتخاب سؤال (جدول را ببینید).
- $\approx \frac{1}{S}\sum_s g(\theta^{(s)})$: انتگرال را با میانگین ساده روی S نمونه‌ی $\theta^{(1)}, \dots, \theta^{(S)}$ جایگزین کن. بالانویس (s) شماره‌ی نمونه است.
- **خطا:** $\mathrm{SD}(g)/\sqrt{S}$. برای نصف کردن خطا چهار برابر نمونه لازم است، هر تعداد پارامتری که داشته باشیم.
- **مثال:** ۱۰،۰۰۰ نمونه از $\mathrm{Beta}(8,4)$؛ ۹،۰۰۰ تای آن‌ها بالای ۵.۰ هستند، پس $P(\theta > 0.5 \mid \mathbf{y}) \approx 0.90$ با خطای Monte Carlo برابر $\sqrt{0.9 \times 0.1/10000} = 0.003$.

۲. زنجیره‌های مارکوف در یک پاراگراف

یک **زنجیره‌ی مارکوف** دنباله‌ای $\theta^{(1)}, \theta^{(2)}, \dots$ است که در آن هر مقدار فقط از مقدار قبلی، با یک قاعده‌ی گذار ثابت، تولید می‌شود. تحت شرایط ملایم، زنجیره نقطه‌ی شروعش را «فراموش می‌کند» و به یک **توزیع ایستا** (stationary) می‌رسد: نسبت بلندمدت زمانی که در هر ناحیه می‌گذراند. MCMC قاعده‌ی گذاری می‌سازد که توزیع ایستایش همان پسین است. دو پیامد: - بخش اولیه‌ی زنجیره هنوز بازتاب نقطه‌ی شروع است. به‌عنوان burn-in (warm-up) دور ریخته می‌شود. - نمونه‌های پشت‌سرهم **همبسته‌اند**، پس S نمونه‌ی همبسته اطلاعات کمتری از S نمونه‌ی مستقل دارند (اندازه‌ی مؤثر نمونه، هفته‌ی ۱۲).

۳. الگوریتم Metropolis

فرض کنید $\tilde{p}(\theta) = p(\mathbf{y} \mid \theta)\, p(\theta)$ پسین **نرمال‌نشده** (درست‌نمایی × پیشین) باشد. یک **توزیع پیشنهادی** متقارن انتخاب کنید، معمولاً یک گام تصادفی نرمال $\theta' \sim \mathcal{N}(\theta, \sigma_{\text{prop}})$.

۱. از یک $\theta^{(0)}$ شروع کنید.

۲. برای $s = 1, 2, \ldots, S$:

- پیشنهاد دهید $\theta' \sim \mathcal{N}(\theta^{(s-1)}, \sigma_{\text{prop}})$.
- احتمال پذیرش را حساب کنید:

$$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')}{\tilde{p}(\theta^{(s-1)})}\right).$$

- $u \sim \text{Uniform}(0,1)$ بکشید. اگر $u < \alpha_{\text{acc}}$، **بپذیرید**: $\theta^{(s)} = \theta'$. وگرنه **رد کنید**: $\theta^{(s)} = \theta^{(s-1)}$ (مقدار فعلی در زنجیره تکرار می‌شود).

چرا به‌طور شهودی کار می‌کند: حرکت‌های سربالا همیشه پذیرفته می‌شوند؛ حرکت‌های سرپایینی با احتمالی برابر نسبت چگالی‌ها پذیرفته می‌شوند. پس زنجیره در هر ناحیه به نسبت احتمال پسینش وقت می‌گذراند.

چرا فقط پسین نرمال‌نشده لازم است: شواهد $p(\mathbf{y})$ هم در صورت و هم در مخرج نسبت ظاهر می‌شود و ساده می‌شود. این کلید سودمندی MCMC است.

جزئیات عملی: - با **لگاریتم** کار کنید: اگر $\ln u < \ln \tilde{p}(\theta') - \ln \tilde{p}(\theta)$ بپذیرید. - **پارامترهای کران‌دار:** اگر یک پیشنهاد خارج از تکیه‌گاه بیفتد (مثلاً $A > 1$)، پیشینش ۰ است، پس $\ln \tilde{p} = -\infty$ و پیشنهاد خودکار رد می‌شود. (راه دیگر نمونه‌گیری روی یک مقیاس تبدیل‌شده و بی‌کران است، مثل هفته‌ی ۳؛ در این صورت باید Jacobian را به لگاریتم پسین اضافه کرد.) - **چند پارامتر:** همه‌ی پارامترها را هم‌زمان با برداری از انحراف معیارهای پیشنهادی پیشنهاد دهید، یا هر بار یک پارامتر را به‌روز کنید (که به نمونه‌گیری Gibbs می‌رسد، هفته‌ی ۱۲).

> **پیشرفته: تعادل جزئی** (detailed balance). یک قاعده‌ی گذار $T(\theta \to \theta')$ توزیع π را ایستا نگه می‌دارد اگر برای همه‌ی جفت‌ها $\pi(\theta)\,T(\theta \to \theta') = \pi(\theta')\,T(\theta' \to \theta)$. این را برای Metropolis با $\tilde{p}(\theta') < \tilde{p}(\theta)$ بررسی کنید: سمت چپ $\pi(\theta)\,q(\theta' \mid \theta)\,\tilde{p}(\theta')/\tilde{p}(\theta)$ و سمت راست $\pi(\theta')\,q(\theta \mid \theta')\cdot 1$ است. با q متقارن و $\pi \propto \tilde{p}$ این دو برابرند.

خواندن ریاضی

- $\tilde{p}(\theta) = p(\mathbf{y} \mid \theta)\,p(\theta)$: درست‌نمایی × prior، بدون تقسیم بر evidence. علامت تیلدا (~) یعنی «نرمال‌نشده».
- $\alpha_{\text{acc}} = \min(1, \tilde{p}(\theta')/\tilde{p}(\theta))$: اگر پیشنهاد معقول‌تر باشد، نسبت بزرگ‌تر از ۱ است و min آن را روی ۱ نگه می‌دارد (همیشه پذیرش). اگر نصف معقول باشد، با احتمال ۵۰٪ پذیرفته می‌شود.
- «$u \sim \text{Uniform}(0,1)$ بکش؛ اگر $u < \alpha$ بود بپذیر» فقط راهی است برای اینکه یک رویداد با احتمال α اتفاق بیفتد (یک سکه‌ی نامتعادل).
- **مثال حل‌شده، قدم ۳:** پیشنهاد ۹۵٪ فقط ۰۵۲٪ برابر ۸۰٪ معقول است؛ $u = 0.30 > 0.052$، پس رد می‌شود و ۸۰٪ بار دوم در زنجیره نوشته می‌شود.

۴. پیشنهادهای نامتقارن Metropolis–Hastings:

اگر توزیع پیشنهادی متقارن نباشد، $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$ (مثلاً پیشنهاد log-normal برای یک نرخ مثبت)، احتمال پذیرش به یک تصحیح نیاز دارد:

$$\alpha_{\text{acc}} = \min\left(1,\ \frac{\tilde{p}(\theta')\,q(\theta \mid \theta')}{\tilde{p}(\theta)\,q(\theta' \mid \theta)}\right).$$

Metropolis حالت خاصی است که جمله‌های q ساده می‌شوند. نمونه‌گیری Gibbs (هفته‌ی ۱۲) حالت خاص دیگری است که در آن هر پیشنهادی پذیرفته می‌شود.

۵. تنظیم و تشخیص مشکلات یک نمونه‌گیر

انحراف معیار پیشنهادی σ_{prop} کارایی را کنترل می‌کند:

σ_{prop}	نرخ پذیرش	رفتار	trace plot چه شکلی است
خیلی کوچک	خیلی بالا (> ۰.۹۰)	گام‌های ریز؛ کاوش کند؛ خودهمبستگی قوی	خطی که آرام سرگردان است
خیلی بزرگ	خیلی پایین (< ۰.۱۰)	بیشتر پیشنهادها در نواحی کم‌احتمال می‌افتند و رد می‌شوند؛ زنجیره گیر می‌کند	پله‌های صاف با فلات‌های طولانی
تقریباً درست	≈ ۰.۲۰ تا ۰.۵۰	اختلاط (mixing) خوب	یک «کرم پشمالو» (fuzzy caterpillar)

نتایج نظری کلاسیک نرخ پذیرش بهینه‌ی حدود ۴۴.۰ را برای یک پارامتر پیشنهاد می‌کنند که برای پارامترهای زیاد به حدود ۲۳.۰ می‌رسد. این‌ها را هدف بدانید، نه قانون. σ_{prop} را در یک اجرای مقدماتی تنظیم کنید (نه در اجرایی که نگه می‌دارید). انحراف معیارهای پیشنهادی خوب هم‌مرتبه با انحراف معیارهای پسین‌اند، و پسین‌های همبسته به پیشنهادهای همبسته (یا reparameterization) نیاز دارند.

Burn-in. بخش اولیه‌ی زنجیره را پیش از رسیدن به ناحیه‌ی پراحتمال دور بریزید؛ trace plot نشان می‌دهد کجا. Thinning (نگه داشتن هر m-امین نمونه) حافظه را کم می‌کند ولی اطلاعات را دور می‌ریزد. برای استنباط درست لازم نیست.

خودهمبستگی. خودهمبستگی با تأخیر h همبستگی بین $\theta^{(s)}$ و $\theta^{(s+h)}$ است. خودهمبستگی بالا یعنی کاوش کند و نمونه‌های مؤثراً مستقل کمتر.

همیشه یک نمونه‌گیر جدید را روی مسئله‌ای با جواب معلوم **اعتبارسنجی کنید** (اینجا، beta-binomial conjugate و شبکه‌ی هفته‌ی ۱۰).

برای یک مهندس

- **انتگرال‌گیری** Monte Carlo همان کاری است که برای تخمین BER یا احتمال قطعی (outage) با شبیه‌سازی می‌کنید: میانگین یک تابع نشانگر روی تعداد زیادی تحقق تصادفی. قانون خطای $1/\sqrt{S}$ همان است، و به همین دلیل نرخ‌های خطای پایین شبیه‌سازی‌های خیلی طولانی لازم دارند.
- یک زنجیره‌ی MCMC یک **فرایند مارکوف** است که توزیع ایستای آن طوری طراحی شده که posterior باشد؛ burn-in همان حالت گذرا و بخش ایستا همان حالت ماندگار است.
- **خودهمبستگی** زنجیره دقیقاً همان خودهمبستگی یک فرایند تصادفی است. نمونه‌های همبسته اطلاعات کمتری دارند، همان‌طور که نمونه‌های نویز همبسته کندتر میانگین‌گیری می‌شوند؛ هفته‌ی ۱۲ این را به «اندازه‌ی مؤثر نمونه» تبدیل می‌کند.
- **تنظیم پیشنهاد** (proposal) یک مسئله‌ی اندازه‌ی گام است، مثل انتخاب اندازه‌ی گام یک فیلتر تطبیقی: خیلی کوچک کند همگرا می‌شود، خیلی بزرگ ناپایدار می‌شود (اینجا: بیشتر قدم‌ها رد می‌شوند).
- تفاوت با شبیه‌سازی معمولی: نمی‌توانیم مستقیم از توزیع هدف نمونه‌ی مستقل تولید کنیم، پس فرایندی می‌سازیم که به آن همگرا شود.

برای مدرس

- **زنجیره را روی تخته اجرا کنید.** منحنی $\theta^7(1-\theta)^3$ را بکشید و چهار قدم مثال حل‌شده را اجرا کنید، در حالی که یک دانشجو مقدارهای «u» را تولید می‌کند (یک اپلیکیشن عدد تصادفی کافی است). قدم ردشده که ۸۰.۰ را تکرار می‌کند

لحظه‌ی کلیدی است؛ بپرسید چرا باید ثبت شود.

- **تشبیهی که جواب می‌دهد:** گردشگری که شب در یک شهر می‌گردد و خیابان‌های روشن را ترجیح می‌دهد. خیابان روشن‌تر: همیشه برو. خیابان تاریک‌تر: فقط گاهی برو. بعد از یک پیاده‌روی طولانی، زمانی که در هر خیابان گذرانده متناسب با روشنایی آن است.
- **سه trace plot را** (خیلی کوچک، خوب، خیلی بزرگ) از بخش D lab کنار هم نشان دهید. دانشجوها این‌طوری در پنج دقیقه خواندن trace plot را یاد می‌گیرند.
- **سؤال‌های رایج.**
  - «چرا فقط از یک *grid* ریز استفاده نکنیم؟» با ۴۰ پارامتر grid ناممکن است؛ هزینه‌ی نمونه‌گیری این‌طور رشد نمی‌کند.
  - «از کجا بدانیم زنجیره همگرا شده؟» این موضوع هفته‌ی ۱۲ است (چند زنجیره، $\hat{R}$).
  - «چرا *burn-in* را دور بریزیم؟» نمونه‌های اولیه هنوز نقطه‌ی شروع دلبخواه را نشان می‌دهند، نه posterior را.

۵.۱.۱ مثال حل‌شده

مدل: یادآوری $y = 7$ از $n = 10$ با پیشین تخت، پس $\tilde{p}(\theta) = \theta^7(1-\theta)^3$ (بدون ثابت‌ها). پسین دقیق: $\mathrm{Beta}(8, 4)$ با میانگین ۶۶۷.۰. پیشنهاد: $\theta' = \theta + \mathcal{N}(0, 0.1)$.

گام	θ فعلی	پیشنهاد θ'	$\tilde{p}(\theta)$	$\tilde{p}(\theta')$	نسبت	u	تصمیم	θ جدید
۱	۵۰.۰	۵۸.۰	۰۰۰۹۷۷.۰	۰۰۱۶۳۶.۰	۶۷۵.۱	۴۲.۰	پذیرش (سربالا)	۵۸.۰
۲	۵۸.۰	۸۰.۰	۰۰۱۶۳۶.۰	۰۰۱۶۷۸.۰	۰۲۶.۱	۷۱.۰	پذیرش (سربالا)	۸۰.۰
۳	۸۰.۰	۹۵.۰	۰۰۱۶۷۸.۰	۰۰۰۰۸۷.۰	۰۵۲.۰	۳۰.۰	**رد**: $u > 0.052$	۸۰.۰
۴	۸۰.۰	۶۶.۰	۰۰۱۶۷۸.۰	۰۰۲۱۴۴.۰	۲۷۸.۱	۸۸.۰	پذیرش (سربالا)	۶۶.۰

زنجیره تا اینجا: ۵۰.۰، ۵۸.۰، ۸۰.۰، **۸۰.۰**، ۶۶.۰. گام ردشده مقدار ۸۰.۰ را در زنجیره تکرار می‌کند. این ضروری است: مقادیر تکراری راهی است که زنجیره ثبت می‌کند ۸۰.۰ از ۹۵.۰ محتمل‌تر است. حذف آن‌ها نمونه را سوگیر می‌کند.

تفسیر. در چهار گام، زنجیره از نقطه‌ی شروع به ناحیه‌ی چگالی پسین بالا (۶.۰ تا ۸.۰) رفت و از پرش به دنباله امتناع کرد. در طول هزاران گام، نسبت زمانی که نزدیک هر مقدار گذرانده می‌شود به $\text{Beta}(8, 4)$ همگرا می‌شود.

۶.۱.۱ lab (۲ ساعت)

اهداف

۱. انتگرال‌گیری Monte Carlo با نمونه‌های مستقل: دیدن خطای $1/\sqrt{S}$.
۲. نوشتن یک نمونه‌گیر Metropolis و اعتبارسنجی آن روی مدل beta-binomial.
۳. نمونه‌گیری از پسین مدل فراموشی توانی، مقایسه با شبکه‌ی هفته‌ی ۱۰، و محاسبه‌ی یک کمیت مشتق‌شده.
۴. تنظیم پیشنهاد: مقایسه‌ی انحراف معیارهای پیشنهادی خیلی کوچک، خوب و خیلی بزرگ با trace plot، نرخ پذیرش و خودهمبستگی.

زمان‌بندی: بخش A ۱۵ دقیقه · بخش B ۲۵ دقیقه · بخش C ۳۵ دقیقه · بخش D ۳۰ دقیقه · بخش E ۱۵ دقیقه.

آماده‌سازی

Code · **کد:** `code/week11.py`، cell ۱ · سلول ۱

بخش A: انتگرال‌گیری Monte Carlo (۱۵ دقیقه)

Code · **کد:** `code/week11.py`، cell ۲ · سلول ۲

خروجی مورد انتظار:

```
Exact: mean = 0.5926, P(theta > 0.65) = 0.1967
S =    100: mean = 0.5865 (MC s.e. 0.0065), P(theta > 0.65) = 0.1800
S =   1000: mean = 0.5935 (MC s.e. 0.0021), P(theta > 0.65) = 0.2160
S =  10000: mean = 0.5937 (MC s.e. 0.0007), P(theta > 0.65) = 0.2022
S = 100000: mean = 0.5927 (MC s.e. 0.0002), P(theta > 0.65) = 0.1971
```

ده برابر نمونه‌ی بیشتر حدود $\sqrt{10} \approx 3.2$ برابر خطای Monte Carlo کمتر می‌دهد.

بخش B: یک نمونه‌گیر Metropolis، اعتبارسنجی‌شده (۲۵ دقیقه)

Code · کد: code/week11.py، cell ۳ · سلول ۳

خروجی مورد انتظار:

```
Acceptance rate: 0.59
Metropolis: mean = 0.5930, SD = 0.0660, 95% interval = [0.458 0.718]
Exact:      mean = 0.5926, SD = 0.0663, 95% interval = [0.46  0.719]
```

بخش C: Metropolis برای مدل فراموشی توانی (۳۵ دقیقه)

همان پیشین‌های هفته‌ی ۱۰: $A \sim \text{Beta}(8, 2)$، $d \sim \text{Gamma}(2, \text{scale } 0.3)$.

Code · کد: code/week11.py، cell ۴ · سلول ۴

خروجی مورد انتظار:

```
Acceptance rate: 0.49
    mean     SD   2.5%  97.5%
A  0.868  0.052  0.762   0.96
d  0.418  0.048  0.325   0.51
Week 10 grid posterior means for comparison: A = 0.867, d = 0.417
```

کمیت‌های مشتق‌شده مجانی به دست می‌آیند. پسین نیمه‌عمر $t_{1/2} = 2^{1/d} - 1$ (هفته‌ی ۲) و یادآوری پیش‌بینی‌شده بعد از ۶۰ روز را حساب کنید:

Code · کد: code/week11.py، cell ۵ · سلول ۵

خروجی مورد انتظار:

```
half-life (days)       mean = 4.493, 95% interval = [2.889 7.443]
P(recall) at 60 days   mean = 0.158, 95% interval = [0.113 0.211]
P(half-life < 3 days | data) = 0.039
```

بخش D: تنظیم توزیع پیشنهادی (۳۰ دقیقه)

Code · کد: code/week11.py، cell ۶ · سلول ۶

خروجی مورد انتظار:

```
too small    proposal SD = [0.002, 0.002]: acceptance = 0.97, lag-10
↪  autocorrelation of d = 0.98, mean of d = 0.473
about right  proposal SD = [0.04, 0.05]: acceptance = 0.50, lag-10
↪  autocorrelation of d = 0.26, mean of d = 0.416
too large    proposal SD = [0.5, 1.5]: acceptance = 0.00, lag-10
↪  autocorrelation of d = 0.94, mean of d = 0.420
```

به این دقت کنید: زنجیره‌ی «خیلی کوچک» تقریباً همه‌چیز را می‌پذیرد ولی می‌خزد (trace) که آرام سرگردان است و خودهمبستگی‌اش حتی با تأخیر ۶۰ نزدیک ۱ است)؛ زنجیره‌ی «خیلی بزرگ» تقریباً همه‌چیز را رد می‌کند و مدت‌های طولانی ساکن می‌ماند (پله‌های صاف). هر دو با ۱۰ ۰۰۰ تکرار برآوردهای غیرقابل اعتماد می‌دهند: میانگین d زنجیره‌ی «خیلی کوچک» (۴۷۳.۰) از میانگین پسین واقعی (۴۱۷.۰) دور است، و میانگین زنجیره‌ی «خیلی بزرگ» فقط به این دلیل نزدیک است که تقریباً از مقدار شروعش (۴۲.۰) تکان نخورده است. زنجیره‌ی خوب تنظیم‌شده شبیه یک کرم پشمالوست و خودهمبستگی‌اش سریع کم می‌شود.

بخش E: تفسیر (۱۵ دقیقه)

در notebook جواب دهید: ۱. آیا خلاصه‌های پسین Metropolis برای (A, d) با شبکه‌ی هفته‌ی ۱۰ توافق دارند؟ این توافق چه چیزی را ثابت می‌کند؟ ۲. نیمه‌عمر حافظه‌ی این شرکت‌کننده را، همراه با عدم قطعیت، در یک جمله چطور گزارش می‌کنید؟ ۳. چرا پیشنهادهای ردشده باید به‌صورت مقادیر تکراری در زنجیره ثبت شوند؟

تمرین‌ها (به ترتیب سختی)

تمرین ۱ ⋆. با نمونه‌گیر بخش B، $P(\theta > 0.65 \mid \mathbf{y})$ را برای ۳۰/۵۰ برآورد کنید و با مقدار دقیق بخش A مقایسه کنید. برای رسیدن به دقت $\pm$۰۱.۰ چند تکرار لازم است؟

تمرین ۲ ⋆. نمونه‌گیر بخش C را از چهار نقطه‌ی شروع خیلی متفاوت اجرا کنید، مثلاً $(0.3, 0.05)$، $(0.99, 2.5)$، $(0.5, 1.0)$ و $(0.9, 0.1)$. همه‌ی traceها را با هم رسم کنید. بعد از چند تکرار از هم قابل تشخیص نیستند؟ (این ایده‌ی پشت R-hat است، هفته‌ی ۱۲.)

تمرین ۳ ⋆⋆ **(مدل نمایی).** `log_post_exp` را با پیشین‌های هفته‌ی ۱۰ بنویسید، از آن نمونه بگیرید و پسین را با شبکه مقایسه کنید. بعد توزیع پیش‌بینی پسین را از نمونه‌هایتان حساب کنید و نمودار PPC هفته‌ی ۱۰ را بازتولید کنید.

تمرین ۴ ⋆⋆ (Metropolis–Hastings) **با پیشنهاد نامتقارن).** از d با یک پیشنهاد گام تصادفی log-normal، $\ln d' \sim \mathcal{N}(\ln d, 0.1)$، که هرگز مقادیر منفی پیشنهاد نمی‌دهد، نمونه بگیرید. تصحیح Hastings یعنی $q(d \mid d')/q(d' \mid d) = d'/d$ را استخراج و اعمال کنید. نشان دهید حذف این تصحیح پسین را سوگیر می‌کند.

تمرین ۵ ⋆⋆⋆ **(پیشرفته: تنظیم تطبیقی).** یک مرحله‌ی warm-up بنویسید که هر ۵۰۰ تکرار انحراف معیارهای پیشنهادی را تنظیم کند تا نرخ پذیرش به سمت ۳۰.۰ برود (مثلاً اگر نرخ خیلی بالاست SD را در ۲.۱ و اگر خیلی پایین است در ۸۰.۰ ضرب کنید)، و بعد آن‌ها را برای اجرای اصلی ثابت کند. چرا تطبیق باید پیش از نمونه‌هایی که نگه می‌دارید متوقف شود؟

lab check-off

نشان دهید: (۱) اعتبارسنجی نمونه‌گیرتان در برابر پسین دقیق Beta (بخش B)، (۲) trace plotها و خلاصه برای مدل توانی همراه با مقایسه با شبکه (بخش C)، و (۳) نمودار تنظیم با نرخ‌های پذیرش (بخش D).

۷.۱.۱ ارتباط با هوش مصنوعی و LLMها

نمونه‌گیری جواب از یک LLM همان Monte Carlo است. چون یک LLM خروجی‌اش را نمونه‌گیری می‌کند (کادر هفته‌ی ۱)، پرسیدن چندباره‌ی یک سؤال جواب‌های مختلفی می‌دهد. ترفند رایجی به نام self-consistency تعداد k جواب نمونه‌گیری می‌کند و رأی اکثریت را می‌گیرد [Wang et al., ۲۰۲۳، ICLR]. آیا کمک می‌کند، و چقدر؟ این یک سؤال Monte Carlo است: دقت رأی اکثریت را به‌راحتی نمی‌توانیم به‌صورت بسته حساب کنیم، ولی می‌توانیم شبیه‌سازی‌اش کنیم و خطای استاندارد Monte Carlo را گزارش کنیم، مثل lab این هفته.

Code ۰ **کد:** `code/week11.py`، cell ۷ ۰ سلول ۷

خروجی مورد انتظار:

```
k =  1 samples: majority-vote accuracy = 0.406 (MCSE 0.003)
k =  5 samples: majority-vote accuracy = 0.506 (MCSE 0.004)
k = 15 samples: majority-vote accuracy = 0.671 (MCSE 0.003)
k = 45 samples: majority-vote accuracy = 0.873 (MCSE 0.002)
```

امتحانش کنید (بدون نیاز به API). ⋆ - `p_answer` را طوری عوض کنید که یک جواب غلط محتمل‌ترین باشد (مثلاً `[0.30,0.35,0.20,0.15]`). حالا رأی اکثریت چه می‌کند؟ چرا؟ - ⋆⋆ برای MCSE برابر ۰۰۱.۰ به چند سؤال شبیه‌سازی‌شده (`n_sim`) نیاز دارید؟ جوابتان را با فرمول $\sqrt{p(1-p)/n}$ بررسی کنید. - ⋆⋆ ارتباط با شناخت: ایده‌ی «خرد جمعی درون فرد» (wisdom of the crowd within) می‌گوید میانگین گرفتن از حدس‌های تکراری یک نفر دقت را بالا می‌برد. کدام فرض این شبیه‌سازی (نمونه‌های مستقل) احتمالاً برای حدس‌های تکراری انسان، یا برای یک LLM با دمای پایین، برقرار نیست؟

۸.۱.۱ اشتباهات رایج و برداشت‌های نادرست

۱. **حذف گام‌های ردشده** به‌جای تکرار مقدار فعلی. این کار نمونه را به سمت دنباله‌ها سوگیر می‌کند.
۲. **استفاده از پسین نرمال‌شده.** لازم نیست و معمولاً ممکن هم نیست؛ شواهد در نسبت ساده می‌شود.
۳. **محاسبه‌ی نسبت بدون لگاریتم.** درست‌نمایی‌ها سرریز می‌کنند. لگاریتم چگالی‌ها را مقایسه کنید.
۴. **فراموش کردن تکیه‌گاه پیشین.** پیشنهادهای خارج از تکیه‌گاه باید $\ln \tilde{p} = -\infty$ بگیرند، وگرنه ممکن است درست‌نمایی در مقادیر ناممکن ارزیابی شود (مثلاً احتمال‌های منفی `nan` تولید می‌کنند).
۵. **قضاوت درباره‌ی نمونه‌گیر فقط با نرخ پذیرش.** نرخ ۳۰٪ همگرایی را تضمین نمی‌کند. به traceها و خودهمبستگی نگاه کنید و چند زنجیره اجرا کنید (هفته‌ی ۱۲).
۶. **نگه داشتن** burn-in یا تنظیم پیشنهاد در طول نمونه‌هایی که نگه می‌دارید. هر دو نتایج را سوگیر می‌کنند.
۷. **«تکرار بیشتر همیشه درستش می‌کند.»** اگر زنجیره در یک مُد گیر کرده یا مقیاس پیشنهاد بد است، تکرار بیشتر همان زنجیره ممکن است در زمان معقول کمکی نکند. اول تشخیص دهید.
۸. **فراموش کردن تصحیح** Hastings با پیشنهادهای نامتقارن.

۹.۱.۱ خودآزمایی

۱. چرا خطای Monte Carlo مستقل از تعداد پارامترهاست، ولی تقریب شبکه‌ای نه؟ **پاسخ** برآورد MC میانگین S نمونه است و خطای استانداردش $\mathrm{SD}/\sqrt{S}$ است، بدون توجه به بُعد θ. یک شبکه باید کل فضای پارامتر را بپوشاند، پس اندازه‌اش با تعداد پارامترها به‌طور نمایی رشد می‌کند.

۲. در الگوریتم Metropolis چرا به شواهد $p(\mathbf{y})$ نیاز نداریم؟ **پاسخ** پذیرش فقط به نسبت چگالی‌های پسین در مقدار پیشنهادی و فعلی بستگی دارد؛ $p(\mathbf{y})$ در هر دو ظاهر می‌شود و ساده می‌شود.

۳. $\tilde{p}(\theta) = 0.004$ فعلی، $\tilde{p}(\theta') = 0.001$ پیشنهادی و $u = 0.18$. پذیرش یا رد؟ اگر $u = 0.31$ باشد چه؟ **پاسخ** نسبت ۲۵.۰ است. با $u = 0.18 < 0.25$ بپذیرید؛ با $u = 0.31 > 0.25$ رد کنید و مقدار فعلی را تکرار کنید.

۴. نرخ پذیرش یک زنجیره ۹۷.۰ است. احتمالاً چه مشکلی هست و trace آن چه شکلی دارد؟ **پاسخ** انحراف معیار پیشنهادی خیلی کوچک است: تقریباً هر گام ریزی پذیرفته می‌شود، زنجیره خیلی آرام کاوش می‌کند و trace مثل خطی است که آرام سرگردان است و خودهمبستگی بالایی دارد. انحراف معیار پیشنهادی را بزرگ کنید.

۵. burn-in چیست و طولش را چطور تعیین می‌کنید؟ **پاسخ** تکرارهای اولیه، پیش از رسیدن زنجیره به ناحیه‌ی پراحتمال، که هنوز بازتاب مقدار شروع دلخواه‌اند. طولش را از روی trace plotها (و از چند زنجیره که از نقاط مختلف شروع شده‌اند) انتخاب کنید و آن تکرارها را دور بریزید.

۶. تصحیح Hastings کی لازم است؟ یک مثال بزنید. **پاسخ** وقتی پیشنهاد نامتقارن است، $q(\theta' \mid \theta) \neq q(\theta \mid \theta')$؛ مثلاً پیشنهاد log-normal برای یک نرخ مثبت، یا پیشنهادی که به موقعیت مقدار فعلی در یک بازه‌ی کران‌دار بستگی دارد.

۷. پسین نیمه‌عمر $t_{1/2} = 2^{1/d} - 1$ را از نمونه‌های d MCMC چطور به دست می‌آورید؟ **پاسخ** هر نمونه‌ی d را به یک نیمه‌عمر تبدیل کنید و نمونه‌های تبدیل‌شده را خلاصه کنید (میانگین، صدک‌ها، .HDI هیچ استخراج اضافه‌ای لازم نیست؛ این یکی از مزیت‌های عملی بزرگ نمونه‌گیری است.

۱۰.۱.۱ تکلیف

این هفته تکلیف نمره‌دار جدیدی نداریم. A4 **(تحلیل بیزی با شبکه)** در ابتدای درس هفته‌ی ۱۲ تحویل داده می‌شود. A5 (MCMC **از صفر)** در هفته‌ی ۱۲ منتشر می‌شود و مستقیماً روی تابع `metropolis` امروز ساخته می‌شود، پس notebook lab را مرتب نگه دارید.

تمرین اختیاری (بدون نمره): تمرین ۲ (چند نقطه‌ی شروع) بهترین آمادگی برای هفته‌ی ۱۲ است.

۱۱.۱.۱ مطالعه‌ی بیشتر

- Kruschke **(۲۰۱۴)، فصل ۷.** Markov chain Monte Carlo: مقدمه‌ای شهودی بر الگوریتم Metropolis (مثال «سیاستمدار»)، منطق آن و نماینده بودن نمونه‌ها. (منبع اصلی.)
- Lewandowsky & Farrell **(۲۰۱۸)، فصل ۷.** برآورد بیزی پارامتر با MCMC، به‌کاررفته روی مدل‌های شناختی.
- Lao & Martin, Kumar **(۲۰۲۱)**: بخش‌های موتورهای استنباط و MCMC [شماره‌ی فصل بررسی شود].

پیش‌نمایش هفته‌ی ۱۲: نمونه‌گیری Gibbs (بدون هیچ ردی)، و سؤالی که هر کاربر MCMC باید جواب دهد: آیا زنجیره‌ام همگرا شده است؟ R-hat و اندازه‌ی مؤثر نمونه را خودمان حساب می‌کنیم و بعد با ArviZ بررسی می‌کنیم.